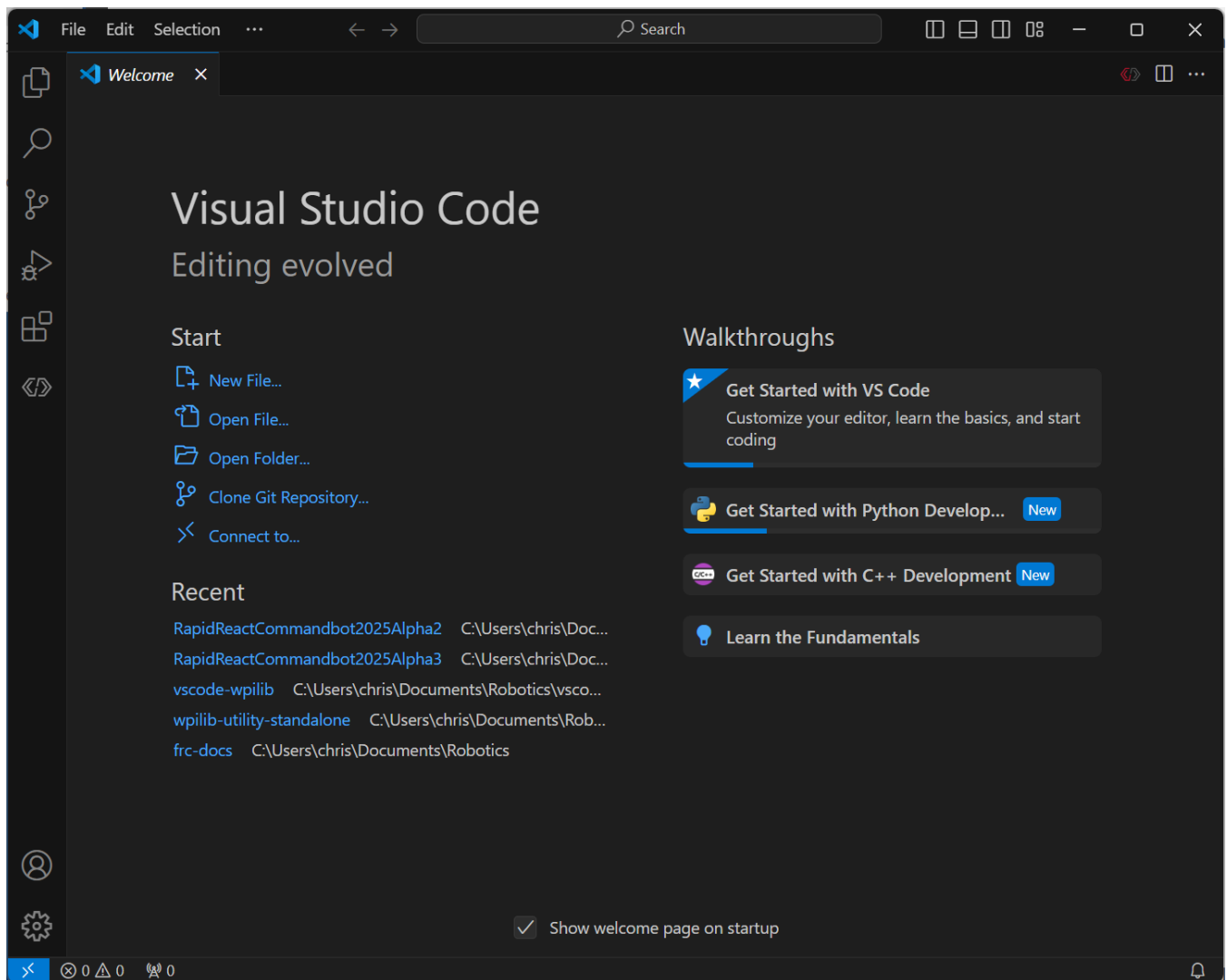


Visual Studio Code Basics and the WPILib Extension / Visual Studio Code 基础与 WPILib 扩展

Microsoft's Visual Studio Code is the supported IDE for Java development in FRC. This article introduces some of the basics of using Visual Studio Code and the WPILib extension.

Microsoft Visual Studio Code 是 FRC 所支持的 Java 开发 IDE。本文介绍 Visual Studio Code 与 WPILib 扩展的一些基本使用方法。

Welcome Page / 欢迎页面



When Visual Studio Code first opens, you are presented with a Welcome page. On this page you will find some quick links that allow you to customize Visual Studio Code as well as a number of links to help documents and videos that may help you learn about the basics of the IDE as well as some tips and tricks.

首次打开 Visual Studio Code 时，你会看到欢迎页面。该页面提供了一些快捷链接，可用于自定义 Visual Studio Code；此外还有许多帮助文档和视频链接，可以帮助你了解 IDE 的基本用法以及一些使用技巧。

You may also notice a small WPILib logo way up in the top right corner. This is one way to access the features provided by the WPILib extension (discussed further below).

你可能还会注意到右上角有一个小型 WPILib 徽标。这是访问 WPILib 扩展所提供功能的一种方式（下文将进一步讨论）。

User Interface / 用户界面

The most important link to take a look at is probably the basic User Interface document. This document describes a lot of the basics of using the UI and provides the majority of the information you should need to get started using Visual Studio Code for FRC.

最值得查看的链接可能是基础用户界面文档。该文档介绍了 UI 的许多基本使用方法，并提供了开始使用 Visual Studio Code 进行 FRC 开发所需的大部分信息。

Command Palette / 命令面板

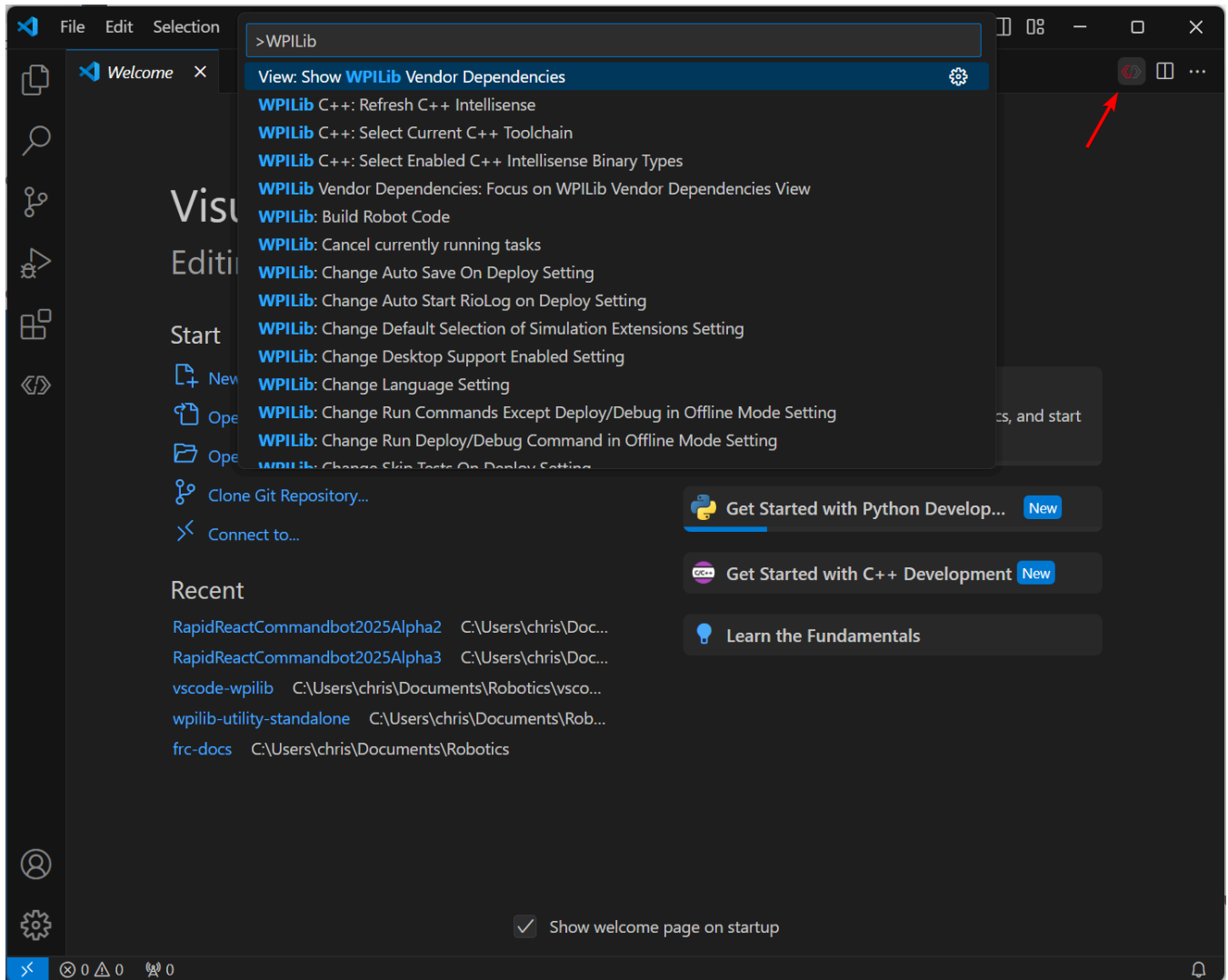
The Command Palette can be used to access or run almost any function or feature in Visual Studio Code (including those from the WPILib extension). The Command Palette can be accessed from the View menu or by pressing Ctrl+Shift+P (Cmd+Shift+P on macOS). Typing text into the window will dynamically narrow the search to relevant commands and show them in the dropdown.

命令面板可用于访问或运行 Visual Studio Code 中几乎所有的功能或特性（包括 WPILib 扩展提供的功能）。可以从 View 菜单访问命令面板，也可以按 Ctrl+Shift+P（macOS 上为 Cmd+Shift+P）将其打开。在窗口中输入文本时，搜索范围会动态缩小到相关命令，并在下拉列表中显示这些命令。

In the following example “wpilib” is typed into the search box after activating the Command Palette, and it narrows the list to functions containing WPILib.

在以下示例中，激活命令面板后，在搜索框中输入了“wpilib”，命令列表因此缩小到名称中包含 WPILib 的功能。

WPILib Extension / WPILib 扩展



The WPILib extension provides the FRC® specific functionality related to creating projects and project components, building, and downloading code to the roboRIO and more. You can access the WPILib commands one of two ways:

WPILib 扩展提供 FRC® 专用功能，包括创建项目和项目组件、构建项目、将代码下载到 roboRIO 等。可以通过以下两种方式之一访问 WPILib 命令：

- By typing “WPILib” into the Command Palette
在命令面板中输入“WPILib”
- By clicking on the WPILib icon in the top right of most windows. This will open the Command Palette with “WPILib” pre-entered
单击大多数窗口右上角的 WPILib 图标。这将打开命令面板，并预先输入“WPILib”

NOTE: It is **not** recommended to install the [Visual Studio IntelliCode](#) plugin with the FRC installation of VS Code as it is known to break IntelliSense in odd ways.

说明：不建议在 FRC 安装版 VS Code 中安装 [Visual Studio IntelliCode](#) 插件，因为已知该插件会以一些异常方式破坏 IntelliSense。

For more information about specific WPILib extension commands, see the other articles in this chapter.

有关特定 WPILib 扩展命令的更多信息，请参阅本章的其他文章。

WPILib Commands in Visual Studio Code / Visual Studio Code 中的 WPILib 命令

This document contains a complete list of the commands provided by the WPILib VS Code Extension and what they do.

本文档完整列出了 WPILib VS Code 扩展提供的命令及其作用。

To access these commands, press Ctrl+Shift+P to open the Command Palette, then begin typing the command name as shown here to filter the list of commands. Click on the command name to execute it.

要访问这些命令，请按 Ctrl+Shift+P 打开命令面板，然后开始输入如下所示的命令名称，以筛选命令列表。单击命令名称即可执行该命令。

- **WPILib: Build Robot Code** - Builds open project using GradleRIO
WPILib: Build Robot Code - 使用 GradleRIO 构建当前打开的项目
- **WPILib: Create a new project** - Create a new robot project
WPILib: Create a new project - 创建新的机器人项目
- **WPILib: Cancel currently running tasks** - Cancel any tasks the WPILib extension is currently running
WPILib: Cancel currently running tasks - 取消 WPILib 扩展当前正在运行的所有任务
- **WPILib: Change Auto Save On Deploy Setting** - Change whether files are saved automatically when doing a Deploy. This defaults to Enabled.
WPILib: Change Auto Save On Deploy Setting - 更改执行 Deploy 时是否自动保存文件。默认为启用。
- **WPILib: Change Auto Start RioLog on Deploy Setting** - Change whether RioLog starts automatically on deploy. This defaults to Enabled.
WPILib: Change Auto Start RioLog on Deploy Setting - 更改部署时是否自动启动 RioLog。默认为启用。
- **WPILib: Change Desktop Support Enabled Setting** - Change whether building robot code on Desktop is enabled. Enable this for test and simulation purposes. This defaults to Desktop Support off.
WPILib: Change Desktop Support Enabled Setting - 更改是否允许在桌面平台上构建机器人代码。若要进行测试和仿真，请启用此项。Desktop Support 默认为关闭。
- **WPILib: Change Language Setting** - Change the language setting for the currently open project.
WPILib: Change Language Setting - 更改当前打开项目所使用的语言。
- **WPILib: Change Run Commands Except Deploy/Debug in Offline Mode Setting** - Change whether GradleRIO is running in Online Mode for commands other then deploy/debug (will attempt to automatically pull dependencies from online). Defaults to enabled (online mode).
WPILib: Change Run Commands Except Deploy/Debug in Offline Mode Setting - 更改执行部署/调试以外的命令时，GradleRIO 是否以 Online Mode 运行（将尝试自动从网络拉取依赖项）。默认为启用（online mode）。
- **WPILib: Change Run Deploy/Debug Command in Offline Mode Setting** - Change whether GradleRIO is running in Online Mode for deploy/debug (will attempt to automatically pull dependencies from online). Defaults to disabled (offline mode).
WPILib: Change Run Deploy/Debug Command in Offline Mode Setting - 更改执行部署/调试时，

GradleRIO 是否以 Online Mode 运行（将尝试自动从网络拉取依赖项）。默认为禁用（offline mode）。

- **WPILib: Change Select Default Simulate Extension Setting** - Change whether simulation extensions are enabled by default (all simulation extensions defined in `build.gradle` will be enabled)

WPILib: Change Select Default Simulate Extension Setting - 更改是否默认启用仿真扩展（将启用 `build.gradle` 中定义的所有仿真扩展）

- **WPILib: Change Skip Tests On Deploy Setting** - Change whether to skip tests on deploy. Defaults to disabled (tests are run on deploy)

WPILib: Change Skip Tests On Deploy Setting - 更改部署时是否跳过测试。默认为禁用（部署时会运行测试）

- **WPILib: Change Stop Simulation on Entry Setting** - Change whether to stop robot code on entry when running simulation. Defaults to disabled (don't stop on entry).

WPILib: Change Stop Simulation on Entry Setting - 更改运行仿真时是否在进入程序时停止机器人代码。默认为禁用（进入时不停止）。

- **WPILib: Change Use WinDbg Preview (From Store) as Windows Debugger Setting** - Change whether to use the VS Code debugger or WinDbg Preview (from Windows Store).

WPILib: Change Use WinDbg Preview (From Store) as Windows Debugger Setting - 更改使用 VS Code 调试器还是 WinDbg Preview（来自 Windows Store）。

- **WPILib: Check for WPILib Updates** - Check for an update to the WPILib GradleRIO version for the project. This does not update the Visual Studio Code extension, tools, or offline dependencies. Users are strongly recommended to use the offline wpilib installer

WPILib: Check for WPILib Updates - 检查项目的 WPILib GradleRIO 版本是否有更新。这不会更新 Visual Studio Code 扩展、工具或离线依赖项。强烈建议用户使用 WPILib 离线安装程序

- **WPILib: Debug Robot Code** - Build and deploy robot code to roboRIO in debug mode and start debugging

WPILib: Debug Robot Code - 以调试模式构建机器人代码并部署到 roboRIO，然后开始调试

- **WPILib: Deploy Robot Code** - Build and deploy robot code to roboRIO

WPILib: Deploy Robot Code - 构建机器人代码并部署到 roboRIO

- **WPILib: Hardware Sim Robot Code** - This builds the current robot code project on your PC and starts it running in simulation using hardware attached to the computer rather than pure software simulation. Requires vendor support.

WPILib: Hardware Sim Robot Code - 在 PC 上构建当前机器人代码项目，并使用连接到计算机的硬件进行仿真，而不是使用纯软件仿真。需要 vendor 支持。

- **WPILib: Import a WPILib 2020-202r Gradle Project** - Open a wizard to help you create a new project from an existing VS Code Gradle project from 2020-2022. Further documentation is in the Importing Last Year's Robot Code section below.

WPILib: Import a WPILib 2020-202r Gradle Project - 打开向导，帮助你基于现有的 2020-2022 VS Code Gradle 项目创建新项目。更多信息请参阅下文的“导入上一年度的机器人代码”章节。

- **WPILib: Install tools from GradleRIO** - Install the WPILib Java tools (e.g. SmartDashboard, Shuffleboard, etc.). Note that this is done by default by the offline installer

WPILib: Install tools from GradleRIO - 安装 WPILib Java 工具（例如 SmartDashboard、Shuffleboard 等）。请注意，离线安装程序默认会执行此操作

- **WPILib: Manage Vendor Libraries** - Install/update 3rd party libraries
WPILib: Manage Vendor Libraries - 安装/更新第三方库
- **WPILib: Open API Documentation** - Opens the WPILib Javadocs documentation
WPILib: Open API Documentation - 打开 WPILib Javadocs 文档
- **WPILib: Open Project Information** - Opens a widget with project information (Project version, extension version, etc.)
WPILib: Open Project Information - 打开一个显示项目信息（项目版本、扩展版本等）的组件
- **WPILib: Open WPILib Command Palette** - This command is used to open a WPILib Command Palette (equivalent of hitting Ctrl+Shift+P and typing `WPILib`)
WPILib: Open WPILib Command Palette - 此命令用于打开 WPILib 命令面板（等同于按 Ctrl+Shift+P 并输入 `WPILib`）
- **WPILib: Open WPILib Help** - This opens a simple page which links to the WPILib documentation (this site)
WPILib: Open WPILib Help - 打开一个简单页面，其中包含指向 WPILib 文档（本站）的链接
- **WPILib: Reset Ask for WPILib Updates Flag** - This will clear the flag on the current project, allowing you to re-prompt to update a project to the latest WPILib version if you previously chose to not update.
WPILib: Reset Ask for WPILib Updates Flag - 清除当前项目上的标志；如果你之前选择不更新，现在可以再次收到将项目更新到最新 WPILib 版本的提示。
- **WPILib: Run a command in Gradle** - This lets you run an arbitrary command in the GradleRIO command environment
WPILib: Run a command in Gradle - 在 GradleRIO 命令环境中运行任意命令
- **WPILib: run Gradle Clean** - Run Gradle Clean to delete build artifacts
WPILib: run Gradle Clean - 运行 Gradle Clean 以删除构建产物
- **WPILib: Set Team Number** - Used to modify the team number associated with a project. This is only needed if you need to change the team number from the one initially specified when creating the project.
WPILib: Set Team Number - 用于修改与项目关联的队伍编号。只有需要更改创建项目时最初指定的队伍编号时，才需要使用此命令。
- **WPILib: Set VS Code Java Home to FRC Home** - Set the VS Code Java Home variable to point to the Java Home discovered by the FRC extension. This is needed if not using the offline installer to make sure the intellisense settings are in sync with the WPILib build settings.
WPILib: Set VS Code Java Home to FRC Home - 将 VS Code Java Home 变量设置为 FRC 扩展发现的 Java Home。如果未使用离线安装程序，则需要执行此操作，以确保 IntelliSense 设置与 WPILib 构建设置保持同步。
- **WPILib: Show Log Folder** - Shows the folder where the WPILib extension stores internal logs. This may be useful when debugging/reporting an extension issue to the WPILib developers
WPILib: Show Log Folder - 显示 WPILib 扩展存储内部日志的文件夹。在调试扩展问题或向 WPILib 开发人员报告问题时，这可能很有用
- **WPILib: Simulate Robot Code** - This builds the current robot code project on your PC and starts it running in simulation. This requires Desktop Support to be set to Enabled.

WPILib: Simulate Robot Code - 在 PC 上构建当前机器人代码项目，并启动项目进行仿真。需要将 Desktop Support 设置为 Enabled。

- **WPILib: Start RioLog** - This starts the RioLog display used to view console output from a robot program

WPILib: Start RioLog - 启动 RioLog 显示界面，用于查看机器人程序的控制台输出

- **WPILib: Start Tool** - This allows you to launch WPILib tools (e.g. SmartDashboard, Shuffleboard, etc.) from inside VS Code

WPILib: Start Tool - 允许你从 VS Code 内启动 WPILib 工具（例如 SmartDashboard、Shuffleboard 等）

- **WPILib: Test Robot Code** - This builds the current robot code project and runs any created tests. This requires Desktop Support to be set to Enabled.

WPILib: Test Robot Code - 构建当前机器人代码项目，并运行已经创建的所有测试。需要将 Desktop Support 设置为 Enabled。

Creating a Robot Program / 创建机器人程序

Once everything is installed, we're ready to create a robot program. WPILib comes with several templates for robot programs. Use of these templates is highly recommended for new users; however, advanced users are free to write their own robot code from scratch.

完成所有安装后，就可以开始创建机器人程序了。WPILib 提供了多个机器人程序模板。强烈建议新用户使用这些模板；高级用户也可以从零开始编写自己的机器人代码。

Choosing a Base Class / 选择基类

To start a project using one of the WPILib robot program templates, users must first choose a base class for their robot. Users subclass these base classes to create their primary `Robot` class, which controls the main flow of the robot program. There are various choices available for the base class:

要使用 WPILib 机器人程序模板之一创建项目，用户必须先为机器人选择基类。用户通过继承这些基类创建主要的 `Robot` 类，由该类控制机器人程序的主要流程。可选择的基类有多种：

TimedRobot / TimedRobot

Documentation: Java

文档: Java

Source: [Java](#)

源码: [Java](#)

The `TimedRobot` class is the base class recommended for most users. It provides control of the robot program through a collection of `init()`, `periodic()`, and `exit()` methods, which are called by WPILib during specific robot states (e.g. autonomous or teleoperated). During these calls, your code typically polls each input device and acts according to the data it receives. For instance, you would typically determine the position of the joystick and state of the joystick buttons on each call and act accordingly. The `TimedRobot` class also provides an example of retrieving autonomous routines through `SendableChooser` (Java)

`TimedRobot` 类是推荐大多数用户使用的基类。它通过一组 `init()`、`periodic()` 和 `exit()` 方法控制机器人程序，这些方法会在特定机器人状态（例如自动阶段或遥控阶段）期间由 WPILib 调用。在这些调用中，代码通常会轮询每个输入设备，并根据收到的数据执行操作。例如，通常会在每次调用时确定操纵杆的位置和操纵杆按钮的状态，然后据此执行操作。`TimedRobot` 类还提供了通过 `SendableChooser` 获取自动例程的示例（Java）。

NOTE: A `TimedRobot Skeleton` template is available that removes some informative comments and the autonomous example. You can use this if you're already familiar with `TimedRobot`. The example shown below is of `TimedRobot Skeleton`.

说明：另有一个 `TimedRobot Skeleton` 模板，它移除了一些说明性注释和自动阶段示例。如果你已经熟悉 `TimedRobot`，可以使用该模板。下面展示的是 `TimedRobot Skeleton` 的示例。

```
import edu.wpi.first.wpilibj.TimedRobot;

/**
 * The methods in this class are called automatically corresponding to each mode, as
 * described in
 * the TimedRobot documentation. If you change the name of this class or the package after
 * creating
 * this project, you must also update the Main.java file in the project.
 */
public class Robot extends TimedRobot {
    /**
     * This function is run when the robot is first started up and should be used for any
     * initialization code.
     */
    public Robot() {}

    @Override
    public void robotPeriodic() {}

    @Override
    public void autonomousInit() {}

    @Override
    public void autonomousPeriodic() {}

    @Override
    public void teleopInit() {}

    @Override
    public void teleopPeriodic() {}

    @Override
    public void disabledInit() {}

    @Override
    public void disabledPeriodic() {}

    @Override
    public void testInit() {}

    @Override
```

```

public void testPeriodic() {}

@Override
public void simulationInit() {}

@Override
public void simulationPeriodic() {}
}

```

Periodic methods are called every 20 ms by default. This can be changed by calling the superclass constructor with the new desired update rate.

默认情况下，periodic 方法每 20 ms 调用一次。可以将期望的新更新周期传给超类构造函数，以更改该周期。

DANGER: Changing your robot rate can cause some unintended behavior (loop overruns). Teams can also use Notifiers to schedule methods at a custom rate.

危险：更改机器人的更新频率可能导致一些意外行为（循环超时）。队伍也可以使用 Notifiers 以自定义频率调度方法。

```

public Robot() {
    super(0.03); // Periodic methods will now be called every 30 ms.
}

```

Timeslice Robot / Timeslice Robot

Documentation: Java

文档: Java

Source: [Java](#)

源码: [Java](#)

The `TimesliceRobot` class extends the `TimedRobot` framework to provide more control over the scheduling of periodic functions. It allows users to allocate specific time slices to different robot operations, running them sequentially within a defined period (typically shorter than `TimedRobot`'s default 20ms). This class is recommended for users who need more precise timing control and consistent starting times for their robot's periodic functions.

`TimesliceRobot` 类扩展了 `TimedRobot` 框架，可以对 periodic 函数的调度进行更精细的控制。它允许用户为不同的机器人操作分配特定时间片，并使这些操作在一个确定的周期内依次运行（该周期通常短于 `TimedRobot` 默认的 20 ms）。对于需要更精确的时序控制，并希望机器人 periodic 函数具有一致启动时间的用户，推荐使用此类。

`TimesliceRobot` provides the same `init()`, `periodic()`, and `exit()` methods as `TimedRobot`, but adds the ability to schedule additional periodic functions with custom allocations. This allows for more efficient use of processing time and can lead to improved performance which can make odometry and estimators more accurate and controller outputs change more consistently.

`TimesliceRobot` 提供与 `TimedRobot` 相同的 `init()`、`periodic()` 和 `exit()` 方法，同时增加了以自定义时间分配调度其他 periodic 函数的能力。这样可以更高效地利用处理时间，并可能提高性能，从而使里程计（odometry）和估算器更加准确，也使控制器输出的变化更加一致。

RobotBase / RobotBase

Documentation: Java

文档: Java

Source: [Java](#)

源码: [Java](#)

The `RobotBase` class is the most minimal base-class offered, and is generally not recommended for direct use. No robot control flow is handled for the user; everything must be written from scratch inside the `startCompetition()` method. The template by default showcases how to process the different operation modes (teleop, auto, etc).

`RobotBase` 类是所提供的最精简基类，通常不建议直接使用。它不会代替用户处理任何机器人控制流程；所有内容都必须在 `startCompetition()` 方法中从零开始编写。默认模板展示了如何处理不同的运行模式（遥控、自动等）。

NOTE: A `RobotBase Skeleton` template is available that offers a blank `startCompetition()` method.

说明: 另有一个 `RobotBase Skeleton` 模板，其中提供了空白的 `startCompetition()` 方法。

Command Robot / Command Robot

The `Command Robot` framework adds to the basic functionality of a `Timed Robot` by automatically polling inputs and converting the raw input data into events. These events are tied to user code, which is executed when the event is triggered. For instance, when a button is pressed, code tied to the pressing of that button is automatically called and it is not necessary to poll or keep track of the state of that button directly. The `Command Robot` framework makes it easier to write compact easy-to-read code with complex behavior, but requires an additional up-front time investment from a programmer in order to understand how the Command Robot framework works.

`Command Robot` 框架在 `Timed Robot` 基础功能之上增加了自动轮询输入并将原始输入数据转换为事件的功能。这些事件与用户代码绑定，并在事件触发时执行这些代码。例如，当按下按钮时，与该按钮按下操作绑定的代码会自动被调用，无需直接轮询或跟踪该按钮的状态。借助 `Command Robot` 框架，可以更轻松地编写紧凑、易读且具有复杂行为的代码，但程序员需要预先投入额外时间来理解 `Command Robot` 框架的工作方式。

Teams using `Command Robot` should see the Command-Based Programming Tutorial.

使用 `Command Robot` 的队伍应参阅“Command-Based 编程教程”。

Romi / Romi

Teams using a Romi should use the `Romi - Timed` or `Romi - Command Bot` template.

使用 Romi 的队伍应使用 `Romi - Timed` 或 `Romi - Command Bot` 模板。

Romi - Timed / Romi - Timed

The `Romi - Timed` template provides a `RomiDrivetrain` class that exposes an `arcadeDrive(double xaxisSpeed, double zaxisRotate)` method. It's up to the user to feed this arcadeDrive function.

`Romi - Timed` 模板提供了一个 `RomiDrivetrain` 类，该类公开 `arcadeDrive(double xaxisSpeed, double zaxisRotate)` 方法。用户需要自行向此 `arcadeDrive` 函数提供输入。

This class also provides functions for retrieving and resetting the Romi's onboard encoders.

此类还提供用于读取和重置 Romi 板载编码器的函数。

Romi - Command Bot / Romi - Command Bot

The `Romi - Command Bot` template provides a `RomiDrivetrain` subsystem that exposes an `arcadeDrive(double xaxisSpeed, double zaxisRotate)` method. It's up to the user to feed this `arcadeDrive` function.

`Romi - Command Bot` 模板提供了一个 `RomiDrivetrain` subsystem，该 subsystem 公开 `arcadeDrive(double xaxisSpeed, double zaxisRotate)` 方法。用户需要自行向此 `arcadeDrive` 函数提供输入。

This subsystem also provides functions for retrieving and resetting the Romi's onboard encoders.

此 subsystem 还提供用于读取和重置 Romi 板载编码器的函数。

Not Using a Base Class / 不使用基类

If desired, users can omit a base class entirely and simply write their program in a `main()` method, as they would for any other program. This is *highly* discouraged - users should not “reinvent the wheel” when writing their robot code - but it is supported for those who wish to have absolute control over their program flow.

如有需要，用户可以完全不使用基类，直接像编写其他程序一样在 `main()` 方法中编写程序。强烈不建议这样做——用户编写机器人代码时不应“重复造轮子”——但对于希望完全控制程序流程的用户，这种方式仍然受到支持。

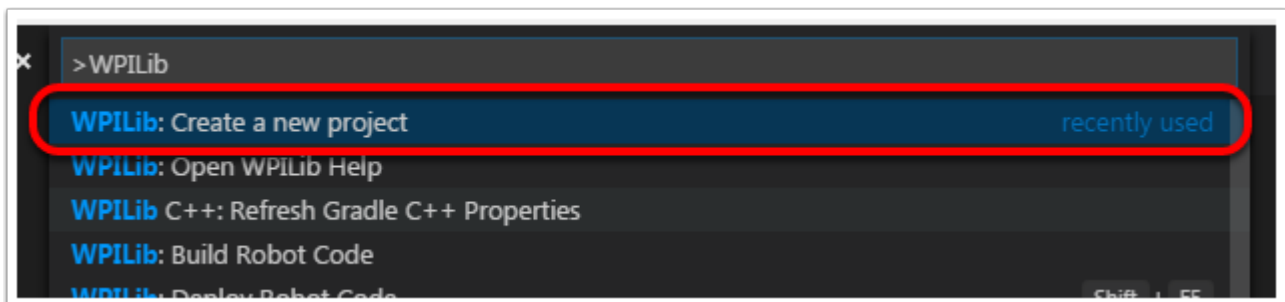
WARNING: Users should *not* modify the `main()` method of a robot program unless they are absolutely sure of what they are doing.

警告：除非用户完全清楚自己在做什么，否则不应修改机器人程序的 `main()` 方法。

Creating a New WPILib Project / 创建新的 WPILib 项目

Once we've decided on a base class, we can create our new robot project. Bring up the Visual Studio Code command palette with `Ctrl+Shift+P`. Then, type “WPILib” into the prompt. Since all WPILib commands start with “WPILib”, this will bring up the list of WPILib-specific VS Code commands. Now, select the **Create a new project** command:

确定基类后，就可以创建新的机器人项目。按 `Ctrl+Shift+P` 打开 Visual Studio Code 命令面板，然后在提示框中输入“WPILib”。由于所有 WPILib 命令都以“WPILib”开头，这将显示 VS Code 中 WPILib 专用命令的列表。现在选择 **Create a new project** 命令：



This will bring up the “New Project Creator Window:”

这将打开“New Project Creator Window”：



The elements of the New Project Creator Window are explained below:

New Project Creator Window 中的各个元素说明如下：

1. **Project Type:** The kind of project we wish to create. This can be an example project, or one of the project templates provided by WPILib. Templates exist for each of the robot base classes. Additionally, a template exists for Command-based projects, which are built on the `TimedRobot` base class but include a number of additional features - this type of robot program is highly recommended for new teams.

Project Type: 希望创建的项目类型。它可以是示例项目，也可以是 WPILib 提供的项目模板之一。每种机器人基类都有对应模板。此外，还提供了 Command-based 项目模板，该模板基于 `TimedRobot` 基类构建，但包含许多额外功能——强烈建议新队伍使用这种机器人程序类型。

2. **Language:** This is the language that will be used for this project. For this document, select Java.

Language: 该项目使用的语言。对于本文档，请选择 Java。

3. **Base Folder:** If this is a template project, this specifies the type of template that will be used.

Base Folder: 如果这是模板项目，此项指定要使用的模板类型。

4. **Project Location:** This determines the folder in which the robot project will be located.

Project Location: 确定机器人项目所在的文件夹。

5. **Project Name:** The name of the robot project. This also specifies the name that the project folder will be given if the Create New Folder box is checked.

Project Name: 机器人项目的名称。如果选中 Create New Folder 复选框，这也会指定项目文件夹的名称。

6. **Create a New Folder:** If this is checked, a new folder will be created to hold the project within the previously-specified folder. If it is *not* checked, the project will be located directly in the previously-specified folder. An error will be thrown if the folder is not empty and this is not checked.

Create a New Folder: 如果选中此项，将在之前指定的文件夹内创建一个新文件夹来存放项目。如果未选中，项目将直接位于之前指定的文件夹中。如果该文件夹非空且未选中此项，将抛出错误。

7. **Team Number:** The team number for the project, which will be used for package names within the project and to locate the robot when deploying code.

Team Number: 项目的队伍编号，用于项目中的 package 名称，并在部署代码时用于查找机器人。

8. **Enable Desktop Support:** Enables unit test and simulation support (see Introduction to Robot Simulation).

Enable Desktop Support: 启用单元测试和仿真支持（请参阅“机器人仿真简介”）。

- **Java:** This option has no effect and can be left checked or unchecked

Java: 此选项不起作用，可以选中，也可以不选中

- **Romi/XRP:** This option has no effect for Romi and XRP templates and can be left checked or unchecked

Romi/XRP: 对于 Romi 和 XRP 模板，此选项不起作用，可以选中，也可以不选中

NOTE: While WPILib fully supports desktop builds, some third-party vendor libraries may not.

说明: 尽管 WPILib 完全支持桌面构建，但某些第三方 vendor library 可能不支持。

Once all the above have been configured, click “Generate Project” and the robot project will be created.

完成上述所有配置后，单击“Generate Project”，机器人项目即会创建。

NOTE: Any errors in project generation will appear in the bottom right-hand corner of the screen.

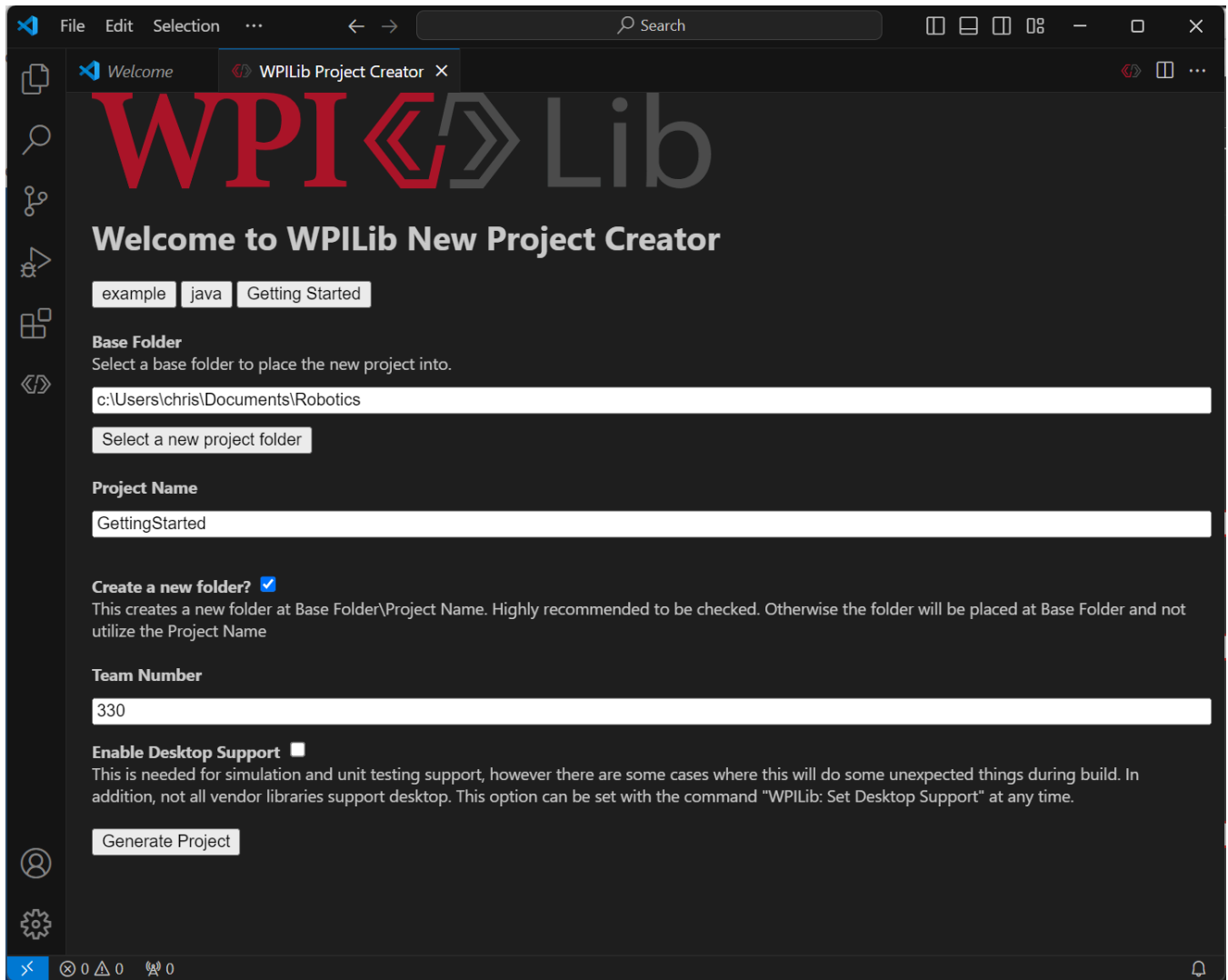
说明: 项目生成过程中的任何错误都会显示在屏幕右下角。

WARNING: Creating projects on OneDrive is not supported as OneDrive’s caching interferes with the build system. Some Windows installations put the Documents and Desktop folders on OneDrive by default.

警告: 不支持在 OneDrive 上创建项目，因为 OneDrive 的缓存会干扰构建系统。某些 Windows 安装会默认将 Documents 和 Desktop 文件夹放在 OneDrive 上。

An example after all options are selected is shown below.

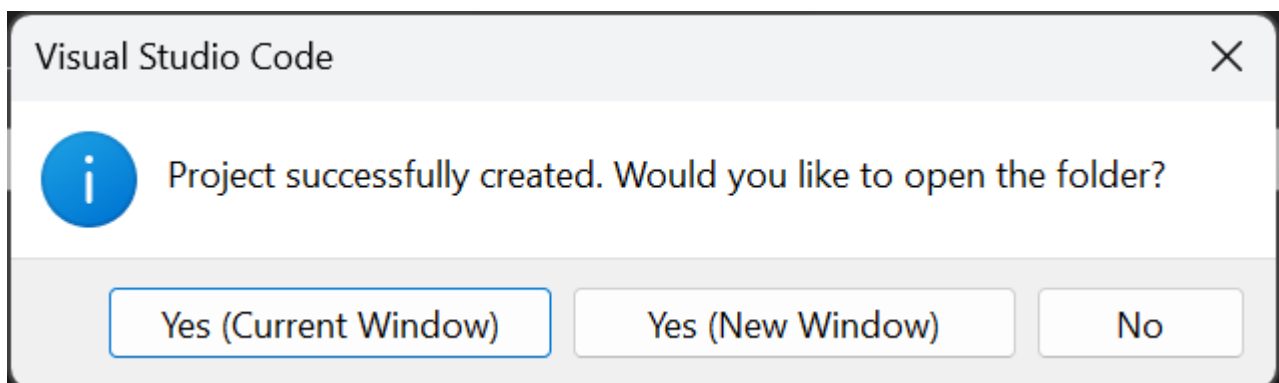
下面展示了选择所有选项后的示例。



Opening The New Project / 打开新项目

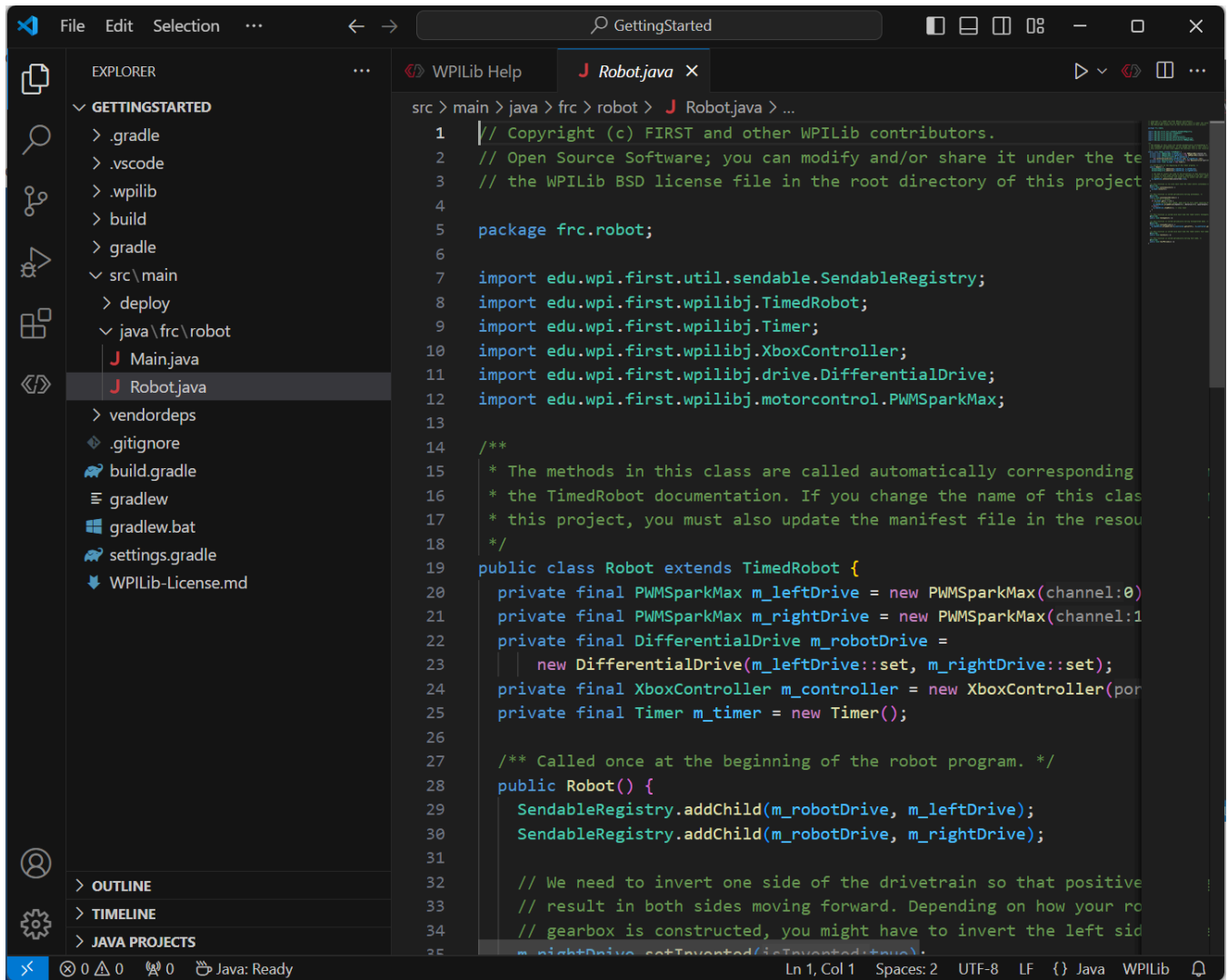
After successfully creating your project, VS Code will give the option of opening the project as shown below. We can choose to do that now or later by typing Ctrl+K then Ctrl+O (or just Command+O on macOS) and select the folder where we saved our project.

成功创建项目后，VS Code 会显示打开该项目的选项，如下所示。可以立即打开，也可以稍后按 Ctrl+K，再按 Ctrl+O（macOS 上只需按 Command+O），然后选择保存项目的文件夹。



Once opened we will see the project hierarchy on the left. Double clicking on the file will open that file in the editor.

打开后，左侧会显示项目层次结构。双击文件即可在编辑器中打开该文件。



3rd Party Libraries / 第三方库

Teams that are using non-PWM motor controllers or advanced sensors will most likely need to install external vendor dependencies.

使用非 PWM 电机控制器或高级传感器的队伍很可能需要安装外部 vendor dependency。

What Are Vendor Dependencies? / 什么是 Vendor Dependency?

A vendor dependency is a way for vendors to add their software library to robot projects. This library can interface with motor controllers and other devices. This way, teams can interact with their devices via CAN and have access to more complex and in-depth features than traditional PWM control.

Vendor dependency 是 vendor 将其软件库添加到机器人项目中的一种方式。该库可以与电机控制器和其他设备进行交互。这样一来，队伍就能通过 CAN 与设备交互，并使用比传统 PWM 控制更复杂、更深入的功能。

Managing Vendor Dependencies / 管理 Vendor Dependency

Vendor dependencies are installed on a per-project basis (so each robot project can have its own set of vendor dependencies). Vendor dependencies can be installed “online” or “offline”. The “online” functionality is done by downloading the dependencies over the internet, while offline is typically provided by a vendor-specific installer.

Vendor dependency 按项目安装（因此每个机器人项目都可以拥有自己的一组 vendor dependency）。Vendor dependency 可以通过“online”或“offline”方式安装。“online”方式通过互联网下载依赖项，而 offline 方式通常由 vendor 专用安装程序提供。

WARNING: If installing a vendor dependency via the “online” mode, make sure to reconnect the computer to the internet and rebuild about every 30 days otherwise the cache will clear, completely deleting the downloaded library install.

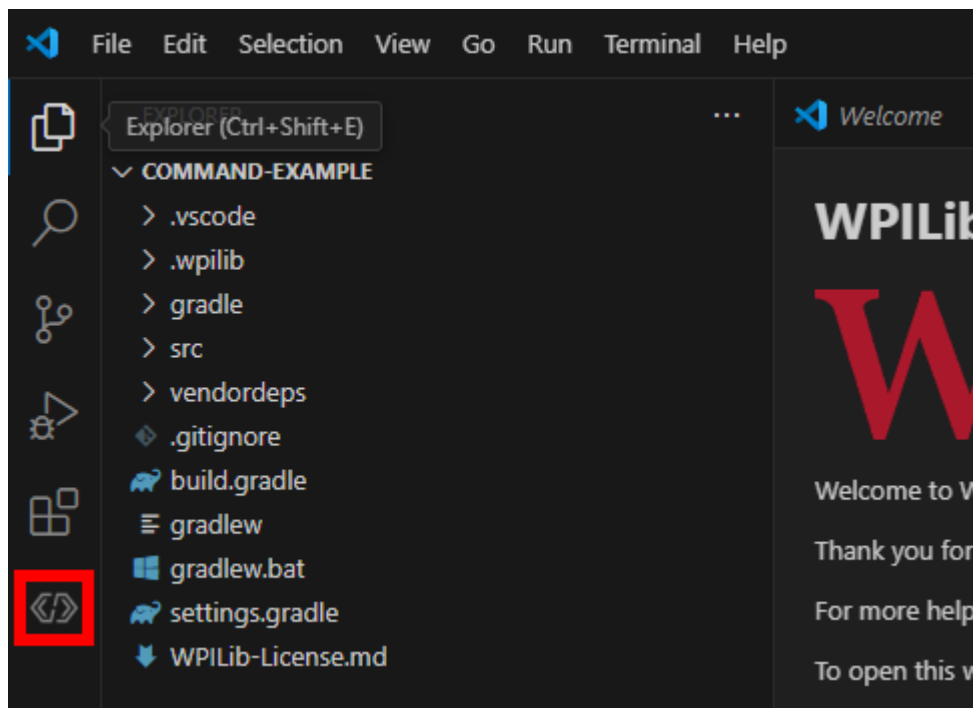
警告：如果通过“online”方式安装 vendor dependency，请确保计算机大约每 30 天重新连接互联网并重新构建一次，否则缓存将被清除，下载的库安装内容也会被完全删除。

NOTE: Vendors recommend using their offline installers when available, because the offline installer is typically bundled with additional programs that are extremely useful when working with their devices.

说明：Vendor 建议在有离线安装程序时使用该程序，因为离线安装程序通常会捆绑一些在使用其设备时非常有用的附加程序。

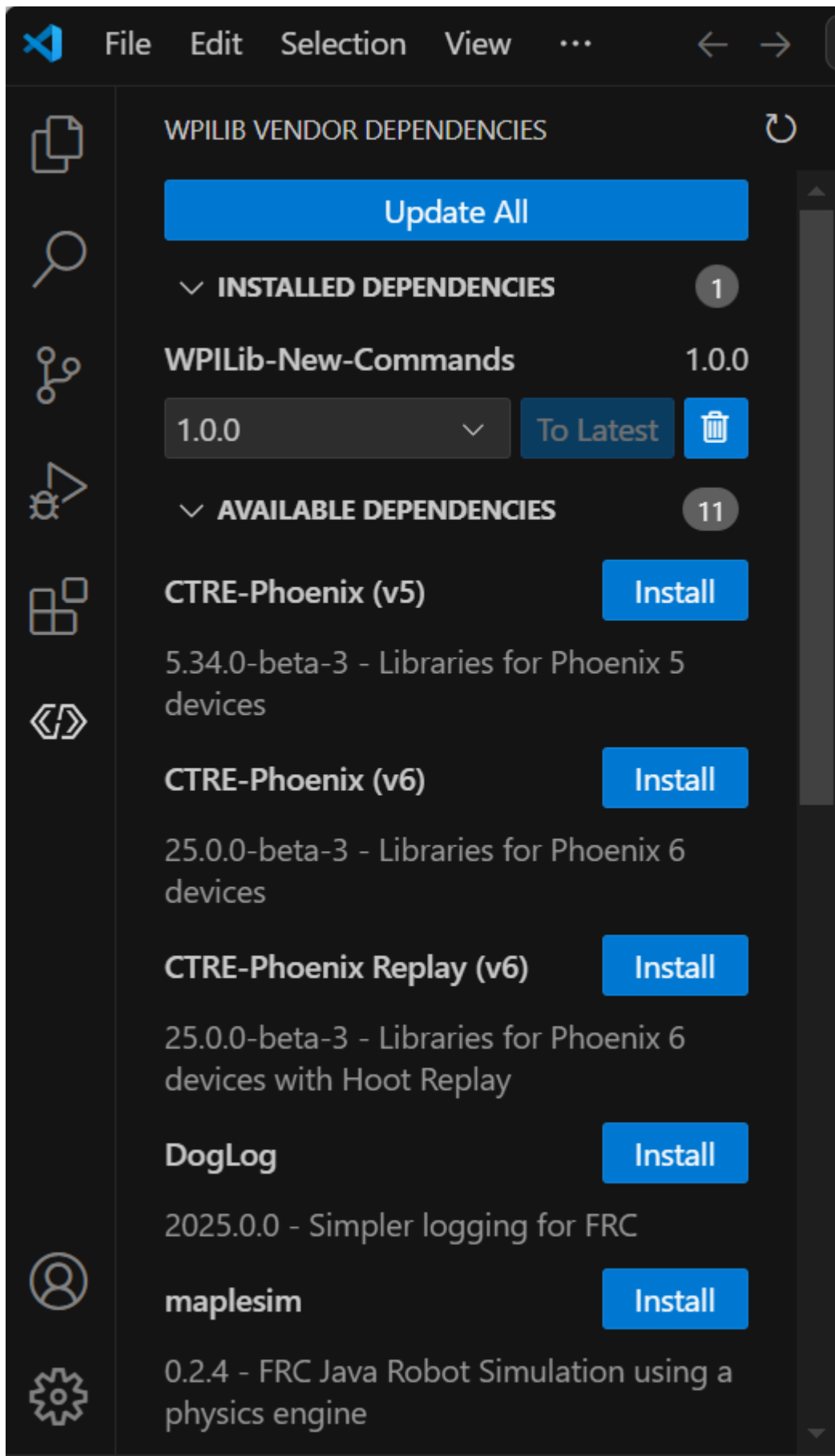
Installing Libraries / 安装库

VS Code



All vendordep operations can be controlled by the Dependency Manager. Click the WPILib logo in the activity bar as shown above to access the interface.

所有 vendordep 操作都可以通过 Dependency Manager 进行控制。单击活动栏中的 WPILib 徽标（如上图所示）即可访问该界面。



Select the desired libraries to add to the project by clicking the **Install** button next to each. The JSON file will be copied to the `vendordeps` folder in the project, adding the library as a dependency to the project.

单击每个所需库旁边的 **Install** 按钮，以选择要添加到项目中的库。JSON 文件会复制到项目中的 `vendordeps` 文件夹，从而将该库作为依赖项添加到项目中。

When an update is available for an installed vendordep you will see the **To Latest** button become available. To update you can either press that or the **Update All** to move all vendordeps to the latest version.

当已安装的 vendordep 有可用更新时，**To Latest** 按钮会变为可用。要进行更新，可以按下该按钮，也可以按 **Update All** 将所有 vendordep 更新到最新版本。

The button with the trash icon will uninstall the vendordep. The dropdown shows what version is currently installed but you can change that to a different version to **update** or **downgrade** to the specified version.

带有垃圾桶图标按钮会卸载 vendordep。下拉列表显示当前安装的版本，也可以将其更改为其他版本，以**更新或降级**到指定版本。

NOTE: The Dependency Manager will automatically build your program when it loses focus. This allows you to use the changed dependencies.

说明：Dependency Manager 失去焦点时会自动构建程序，以便你使用更改后的依赖项。

How Does It Work? / 工作原理

For Java, a JSON file describing the vendor library is installed on your system to `~/wpilib/YYYY/vendordeps` (where YYYY is the year and ~ is `C:\Users\Public` on Windows). This is often done by an offline installer, but may need to be done manually if a `.zip` of the `.json` files is provided. This file is then used from VS Code to add to the library to each individual project. Vendor library information is managed on a per-project basis to make sure that a project is always pointing to a consistent version of a given vendor library. The libraries themselves are placed in the Maven cache at `C:\Users\Public\wpilib\YYYY\maven`. Vendors can place a local copy here with an offline installer (recommended) or require users to be connected to the internet for an initial build to fetch the library from a remote Maven location.

对于 Java，描述 vendor library 的 JSON 文件会安装到系统的 `~/wpilib/YYYY/vendordeps` 中（YYYY 表示年份；在 Windows 上，~ 为 `C:\Users\Public`）。这通常由离线安装程序完成，但如果 vendor 提供的是包含 `.json` 文件的 `.zip`，则可能需要手动完成。随后，可以在 VS Code 中使用该文件，将库添加到每个单独的项目。Vendor library 信息按项目管理，以确保项目始终指向指定 vendor library 的一致版本。库本身放置在 `C:\Users\Public\wpilib\YYYY\maven` 的 Maven 缓存中。Vendor 可以通过离线安装程序（推荐）在此处放置本地副本，也可以要求用户在首次构建时连接互联网，从远程 Maven 位置获取该库。

This JSON file allows specification of complex libraries with multiple components (Java, JNI, etc.) and also helps handle some complexities related to simulation.

此 JSON 文件允许指定包含多个组件（Java、JNI 等）的复杂库，也有助于处理与仿真相关的一些复杂情况。

Building and Deploying Robot Code / 构建并部署机器人代码

Robot projects must be compiled (“built”) and deployed in order to run on the roboRIO. Since the code is not compiled natively on the robot controller, this is known as “cross-compilation.”

机器人项目必须先编译（“构建”）并部署，才能在 roboRIO 上运行。由于代码不是直接在机器人控制器上进行原生编译，因此这一过程称为“交叉编译”。

To build and deploy a robot project, do one of:

要构建并部署机器人项目，请执行以下操作之一：

1. Open the Command Palette and enter/select “Build Robot Code”

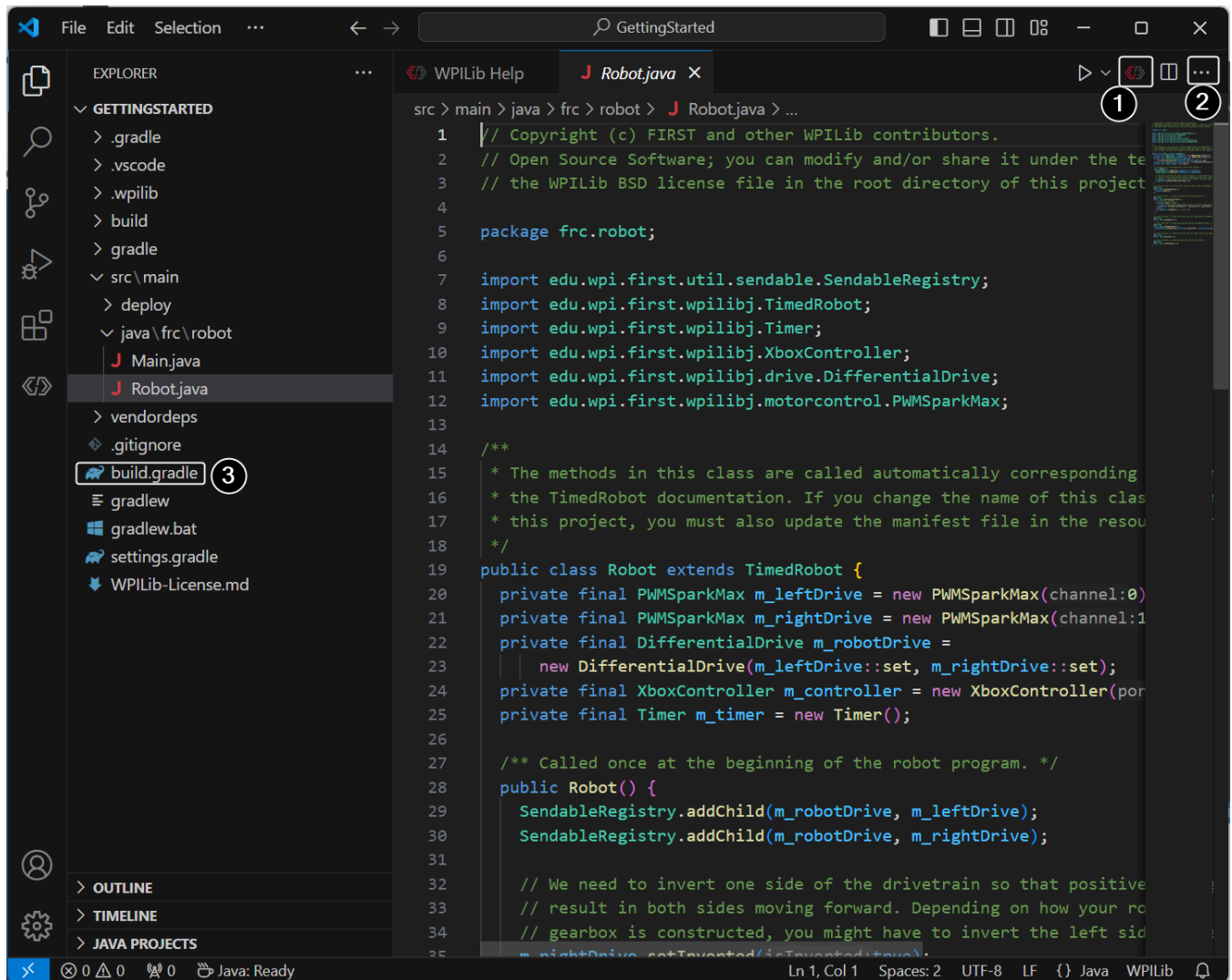
打开命令面板，然后输入/选择“Build Robot Code”

2. Open the shortcut menu indicated by the ellipses in the top right corner of the VS Code window and select “Build Robot Code”

打开 VS Code 窗口右上角省略号所指示的快捷菜单，然后选择“Build Robot Code”

3. Right-click on the build.gradle file in the project hierarchy and select “Build Robot Code”

在项目层次结构中右键单击 build.gradle 文件，然后选择“Build Robot Code”



Deploy robot code by selecting “Deploy Robot Code” from any of the three locations from the previous instructions. That will build (if necessary) and deploy the robot program to the roboRIO.

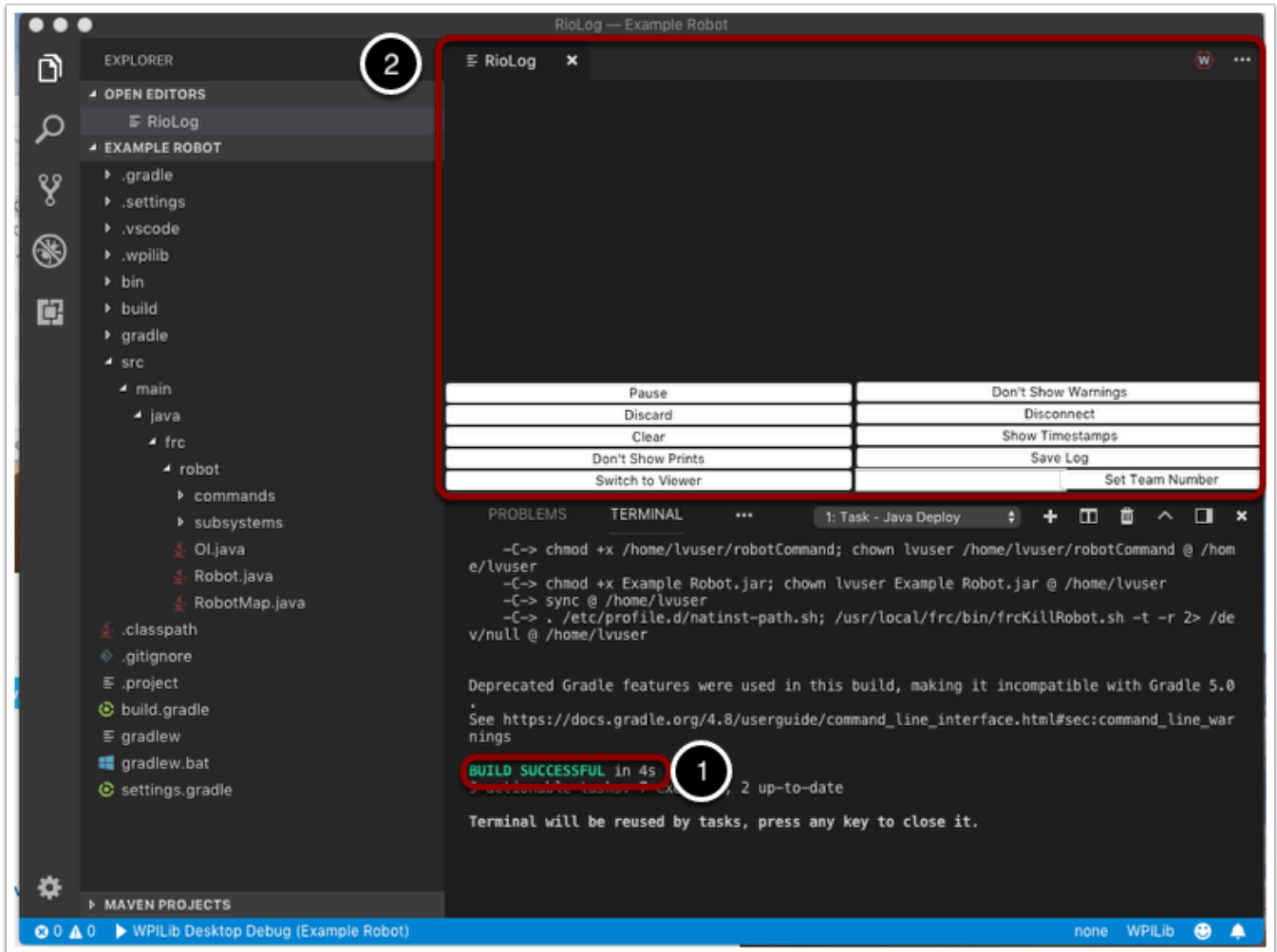
从上述三个位置中的任意一个选择“Deploy Robot Code”即可部署机器人代码。该操作会构建机器人程序（如有必要），然后将其部署到 roboRIO。

WARNING: Avoid powering off the robot while deploying robot code. Interrupting the deployment process can corrupt the roboRIO filesystem and prevent your code from working until the roboRIO is re-imaged.

警告：部署机器人代码时，请避免关闭机器人电源。中断部署过程可能损坏 roboRIO 文件系统，导致代码无法工作，直至重新为 roboRIO 写入镜像。

If successful, we will see a “Build Successful” message (1) and the RioLog will open with the console output from the robot program as it runs (2).

如果成功，将看到“Build Successful”消息 (1)，同时 RioLog 会打开，并在机器人程序运行时显示其控制台输出 (2)。



Viewing Console Output / 查看控制台输出

For viewing the console output of text based programs the roboRIO implements a NetConsole. There are two main ways to view the NetConsole output from the roboRIO: The Console Viewer in the FRC Driver Station and the Riolog plugin in VS Code.

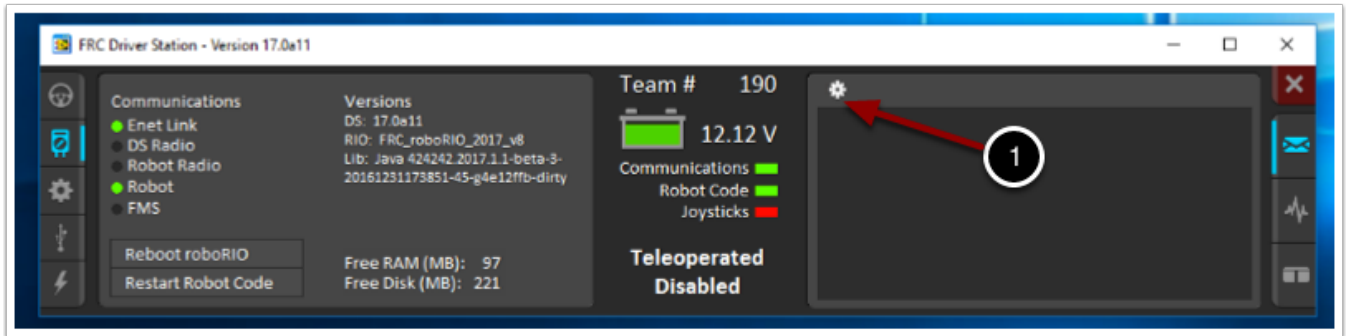
为了查看文本程序的控制台输出，roboRIO 实现了 NetConsole。查看 roboRIO 的 NetConsole 输出主要有两种方式：FRC Driver Station 中的 Console Viewer，以及 VS Code 中的 Riolog 插件。

NOTE: On the roboRIO, the NetConsole is only for program output. If you want to interact with the system console you will need to use SSH or the Serial console.

说明：在 roboRIO 上，NetConsole 仅用于程序输出。如果希望与系统控制台交互，需要使用 SSH 或 Serial console。

Console Viewer / Console Viewer

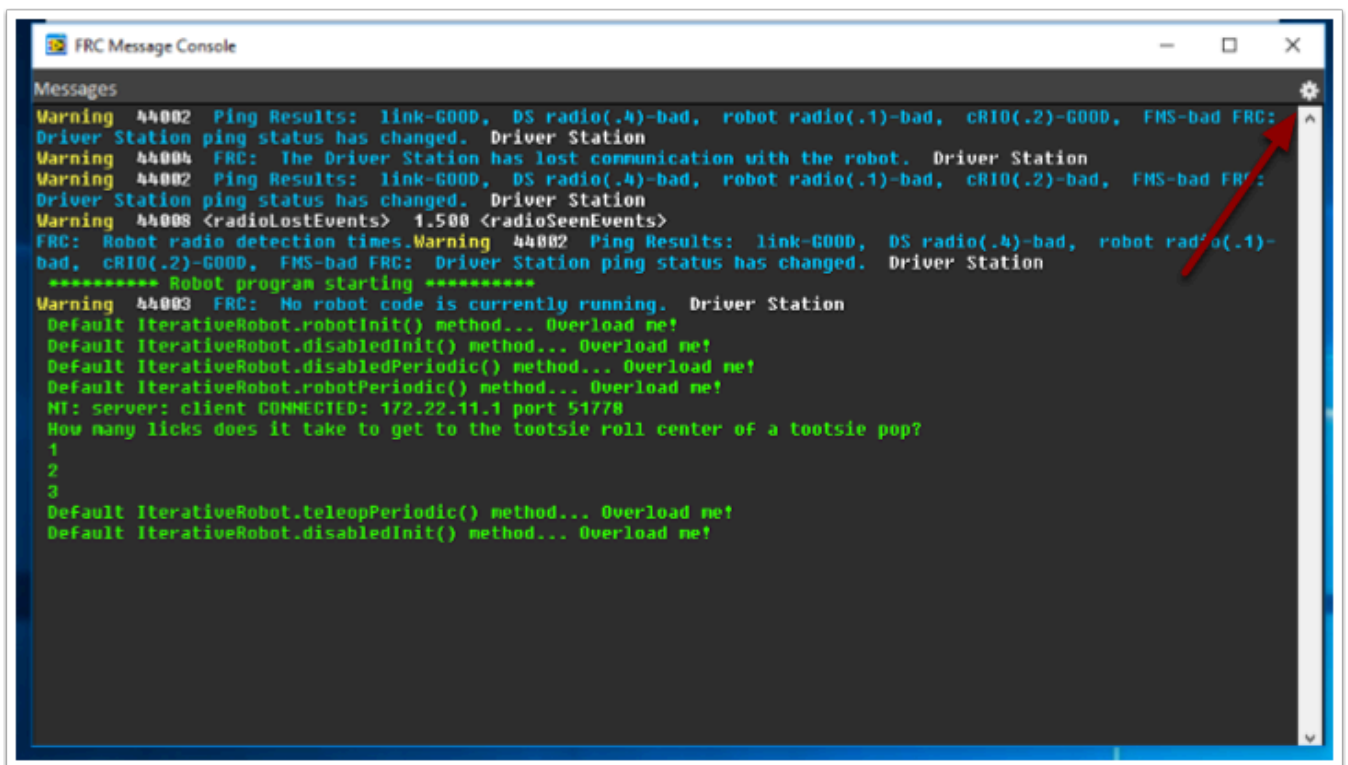
Opening the Console Viewer / 打开 Console Viewer



To open Console Viewer, first open the FRC® Driver Station. Then, click on the gear at the top of the message viewer window (1) and select “View Console”.

要打开 Console Viewer，首先打开 FRC® Driver Station。然后单击消息查看器窗口顶部的齿轮 (1)，并选择“View Console”。

Console Viewer Window / Console Viewer 窗口



The Console Viewer window displays the output from our robot program in green. The gear in the top right can clear the window and set the level of messages displayed.

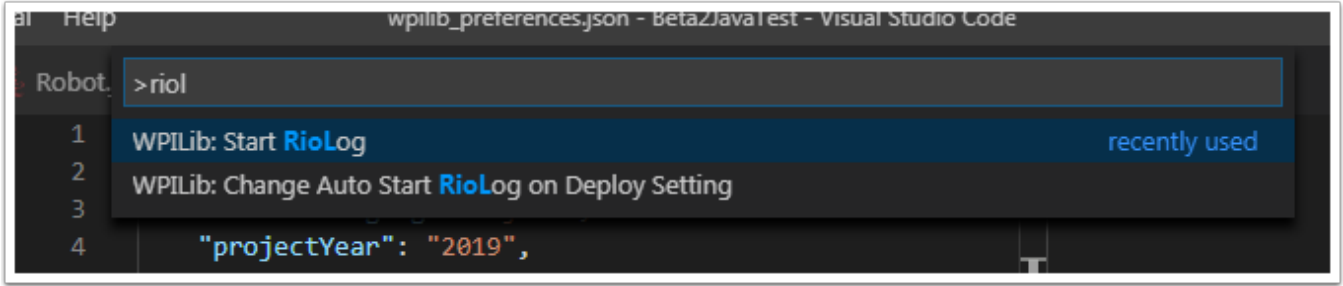
Console Viewer 窗口以绿色显示机器人程序的输出。右上角的齿轮可用于清空窗口以及设置显示的消息级别。

Riolog VS Code Plugin / Riolog VS Code 插件

The Riolog plugin is a VS Code view that can be used to view the NetConsole output in VS Code (credit for the original Eclipse version: Manuel Stoeckl, FRC1511).

RioLog 插件是 VS Code 中的一个视图，可用于在 VS Code 中查看 NetConsole 输出（原 Eclipse 版本作者：Manuel Stoeckl，FRC1511）。

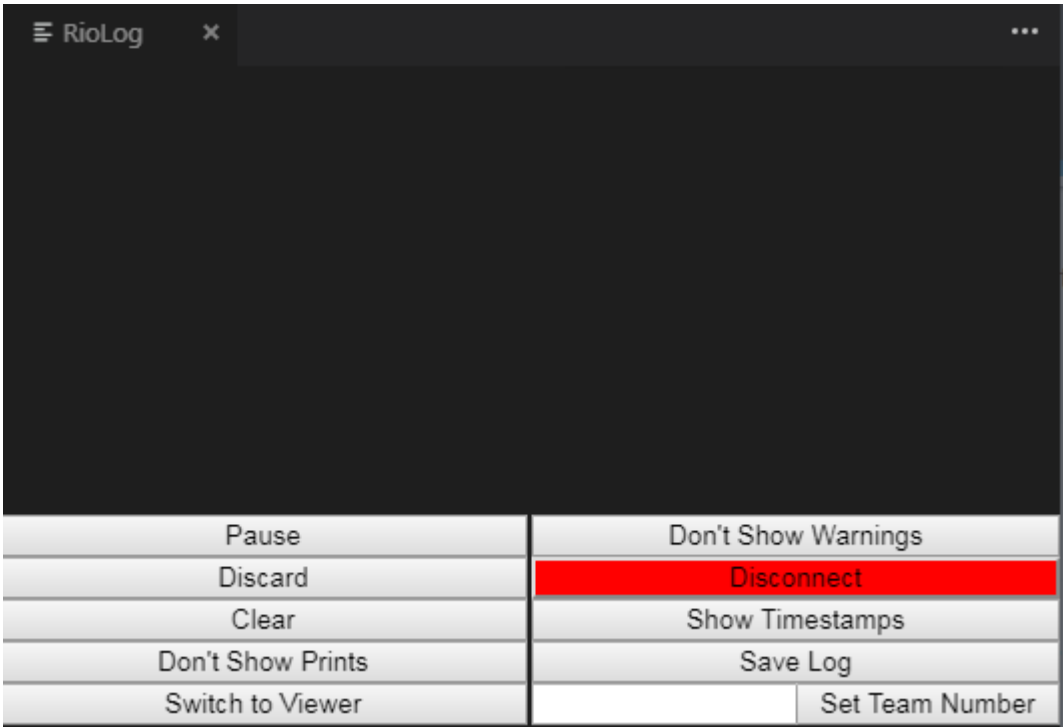
Opening the RioLog View / 打开 RioLog 视图



By default, the RioLog view will open automatically at the end of each roboRIO deploy. To launch the RioLog view manually, press Ctrl+Shift+P to open the command palette and start typing “RioLog”, then select the WPILib: Start RioLog option.

默认情况下，每次完成向 roboRIO 的部署后，RioLog 视图都会自动打开。要手动启动 RioLog 视图，请按 Ctrl+Shift+P 打开命令面板，开始输入“RioLog”，然后选择 WPILib: Start RioLog 选项。

RioLog Window / Riolog 窗口



The RioLog view should appear in the top pane. The Riolog contains a number of controls for manipulating the console:

RioLog 视图应显示在顶部窗格中。Riolog 包含许多用于操作控制台的控件：

- **Pause/Resume Display** - This will pause/resume the display. In the background, the new packets will still be received and will be displayed when the resume button is clicked.
Pause/Resume Display - 暂停/恢复显示。在后台仍会接收新数据包，并在单击恢复按钮时显示这些数据包。
- **Discard/Accept Incoming** - This will toggle whether to accept new packets. When packets are being

discarded the display will be paused and all packets received will be discarded. Clicking the button again will resume receiving packets.

Discard/Accept Incoming - 切换是否接受新数据包。丢弃数据包时，显示将暂停，并且所有收到的数据包都会被丢弃。再次单击该按钮将恢复接收数据包。

- **Clear** - This will clear the current contents of the display.

Clear - 清除当前显示的内容。

- **Don't Show/Show Prints** - This shows or hides messages categorized as print statements

Don't Show/Show Prints - 显示或隐藏归类为 print 语句的消息

- **Switch to Viewer** - This switches to viewer for saved log files

Switch to Viewer - 切换到已保存日志文件的查看器

- **Don't Show/Show Warnings** - This shows or hides messages categorized as warnings

Don't Show/Show Warnings - 显示或隐藏归类为警告的消息

- **Disconnect/Reconnect** - This disconnects or reconnects to the console stream

Disconnect/Reconnect - 断开与控制台数据流的连接，或重新连接

- **Show/Don't Show Timestamps** - Shows or hides timestamps on messages in the window

Show/Don't Show Timestamps - 显示或隐藏窗口中消息的时间戳

- **Save Log** - Copies the log contents into a file you can save and view or open later with the RioLog viewer (see Switch to Viewer above)

Save Log - 将日志内容复制到文件中，以便保存并查看，或稍后使用 RioLog viewer 打开（参阅上面的 Switch to Viewer）

- **Set Team Number** - Sets the team number of the roboRIO to connect to the console stream on, set automatically if RioLog is launched by the deploy process

Set Team Number - 设置要连接其控制台数据流的 roboRIO 的队伍编号；如果 RioLog 由部署过程启动，则会自动设置

Debugging a Robot Program / 调试机器人程序

Inevitably, a program will not behave in the way we expect it to behave. When this occurs, it becomes necessary to figure out why the program is doing what it is doing, so that we can make it do what we want it to do, instead. Such an undesired program behavior is called a “bug,” and this process is called “debugging.”

程序不可避免地会出现不符合预期的行为。发生这种情况时，需要确定程序为何会以当前方式运行，才能使它按我们期望的方式运行。这种非预期的程序行为称为“bug”，查找并解决其原因的过程称为“debugging”。

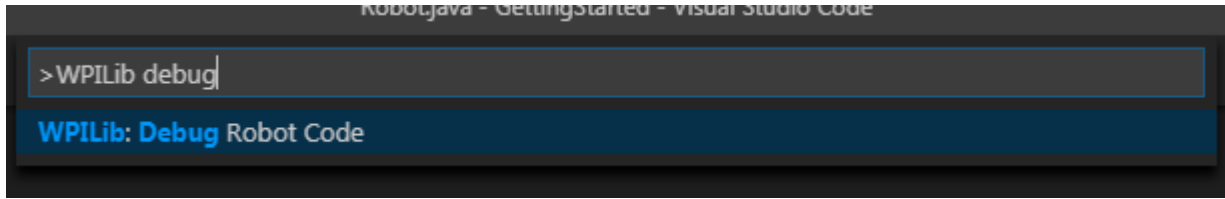
A debugger is a tool used to control program flow and monitor variables in order to assist in debugging a program. This section will describe how to set up a debug session for an FRC® robot program.

Debugger 是一种用于控制程序流程并监视变量的工具，可协助调试程序。本节将介绍如何为 FRC® 机器人程序设置调试会话。

NOTE: For beginning users who need to debug their programs but do not know/have time to learn how to use a debugger, it is often possible to debug a program simply by printing the relevant program state to the console. However, it is strongly recommended that students eventually learn to use a debugger.

说明：对于需要调试程序，但不知道如何使用 debugger 或没有时间学习的初学者，通常可以通过将相关程序状态打印到控制台来调试程序。不过，仍强烈建议学生最终学会使用 debugger。

Running the Debugger / 运行 Debugger



Press Ctrl+Shift+P and type `WPILib` or click on the **WPILib Menu Item** to open the Command palette with WPILib pre-populated. Type Debug and select the Debug Robot Code menu item to start debugging. The code will download to the roboRIO and begin debugging.

按 Ctrl+Shift+P 并输入 `WPILib`，或单击 **WPILib Menu Item**，打开已预先填入 WPILib 的命令面板。输入 Debug，然后选择 Debug Robot Code 菜单项以开始调试。代码会下载到 roboRIO，并开始调试。

Breakpoints / 断点

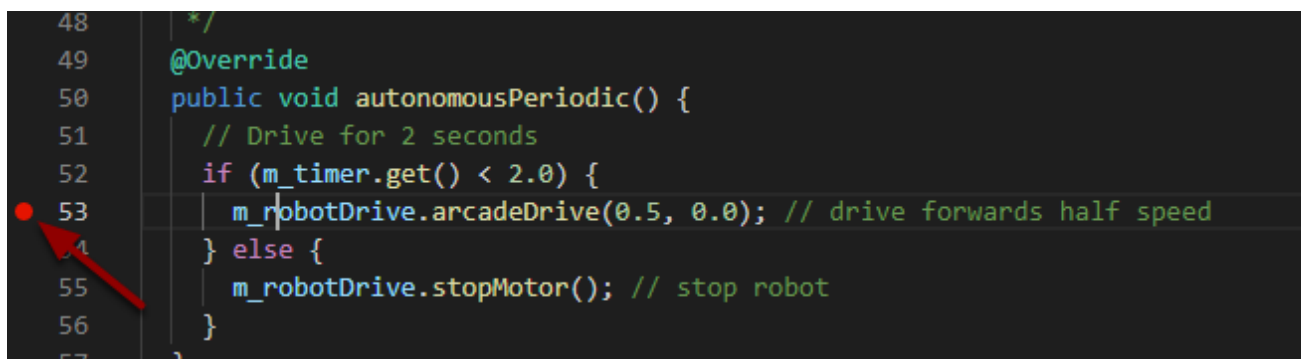
A “breakpoint” is a line of code at which the debugger will pause the program execution so that the user can examine the program state. This is extremely useful while debugging, as it allows the user to pause the program at specific points in problematic code to determine where exactly the program is deviating from the expected behavior.

“断点”是一行代码；程序执行到该行时，debugger 会暂停程序，使用户能够检查程序状态。这对调试非常有用，因为用户可以在有问题的代码中的特定位置暂停程序，以确定程序究竟从何处开始偏离预期行为。

The debugger will automatically pause at the first breakpoint it encounters.

Debugger 会在遇到的第一个断点处自动暂停。

Setting a Breakpoint / 设置断点



Click in the left margin of the source code window (to the left of the line number) to set a breakpoint in your user program: A small red circle indicates the breakpoint has been set on the corresponding line.

单击源代码窗口的左侧边距（行号左侧），即可在用户程序中设置断点：一个小红圈表示已在对应行设置断点。

Debugging with Print Statements / 使用 Print 语句进行调试

Another way to debug your program is to use print statements in your code and view them using the RioLog in Visual Studio Code or the Driver Station. Print statements should be added with care as they are not very efficient especially when used in high quantities. They should be removed for competition as they can cause loop overruns.

另一种调试程序的方法是在代码中使用 print 语句，并通过 Visual Studio Code 中的 RioLog 或 Driver Station 查看这些语句的输出。添加 print 语句时应谨慎，因为它们的效率不高，尤其是在大量使用时。比赛前应将其移除，因为它们可能导致循环超时。

```
System.out.print("example");
```

Debugging with NetworkTables / 使用 NetworkTables 进行调试

NetworkTables can be used to share robot information with your debugging computer. NetworkTables can be viewed with your favorite Dashboard or OutlineViewer. One advantage of NetworkTables is that tools like Shuffleboard can be used to graphically analyze the data. These same tools can then be used with same data to later provide an operator interface for your drivers.

NetworkTables 可用于与调试计算机共享机器人信息。可以使用你常用的 Dashboard 或 OutlineViewer 查看 NetworkTables。NetworkTables 的一个优点是，可以使用 Shuffleboard 等工具以图形方式分析数据。之后，还可以使用这些工具和相同的数据，为 Driver 提供操作界面。

Learn More / 了解更多

- To learn more about debugging with VS Code see this [link](#).

要进一步了解如何使用 VS Code 进行调试，请参阅此[链接](#)。

- Some of the features mentioned in this VS Code [article](#) will help you understand and diagnose problems with your code. The Quick Fix (yellow light bulb) feature can be very helpful with a variety of problems including what to import.

此 VS Code [文章](#)中提到的一些功能可帮助你理解和诊断代码问题。Quick Fix（黄色灯泡）功能可用于解决多种问题，包括确定需要 import 的内容。

- One of the best ways to prevent having to debug so many issues is to do Unit Testing.

避免频繁调试问题的最佳方法之一是进行单元测试。

- Verifying that your robot works in Simulation is also a great way to prevent having to do complex debugging on the actual robot.

在仿真中验证机器人是否正常工作，也是避免在真实机器人上进行复杂调试的好方法。

Importing Last Year's Robot Code / 导入上一年度的机器人代码

Due to changes in the project, it is necessary to update the build files for a previous years Gradle project. It is also necessary to import vendor libraries again, since last year's vendor libraries must be updated to be compatible with this year's projects.

由于项目发生了变化，需要更新上一年度 Gradle 项目的构建文件。还需要重新导入 vendor library，因为上一年度的 vendor library 必须更新，才能与本年度项目兼容。

Automatic Import / 自动导入

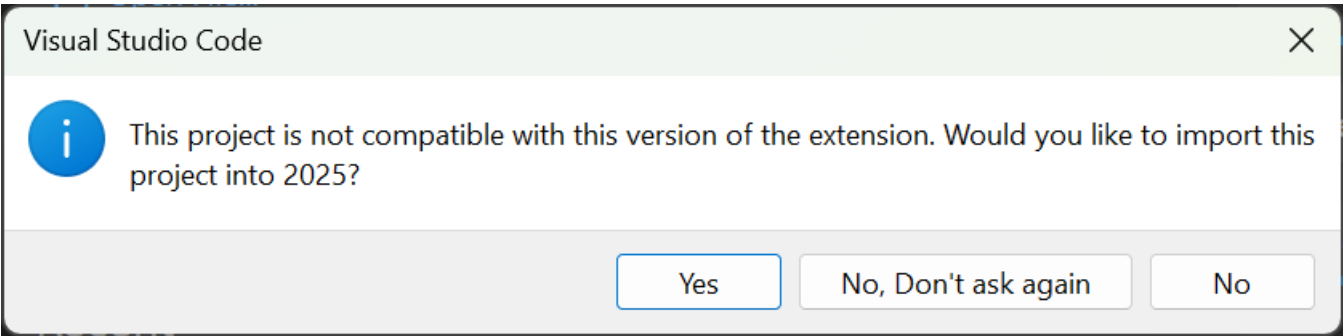
To make it easy for teams to import previous years gradle projects into the current year's framework, WPILib includes a wizard for importing previous years projects into VS Code. This will generate the necessary gradle components and load the project into VS Code. In place upgrades are not supported.

为了方便队伍将往年 Gradle 项目导入本年度框架，WPILib 提供了一个向导，用于将往年项目导入 VS Code。该向导会生成必要的 Gradle 组件，并将项目加载到 VS Code 中。不支持原地升级。

IMPORTANT: The import process copies your project source files from the current directory to a new directory and completely regenerates the gradle files. If you made non-standard updates to the `build.gradle`, you will need to make those changes again. For this reason, in place upgrades are not supported. It is also necessary to import vendor libraries again, since last year's vendor libraries must be updated to be compatible with this year's projects.

重要： 导入过程会将项目源文件从当前目录复制到新目录，并完全重新生成 Gradle 文件。如果对 `build.gradle` 进行了非标准修改，则需要再次进行这些修改。因此，不支持原地升级。还需要重新导入 vendor library，因为上一年度的 vendor library 必须更新，才能与本年度项目兼容。

Launching the Import Wizard / 启动导入向导

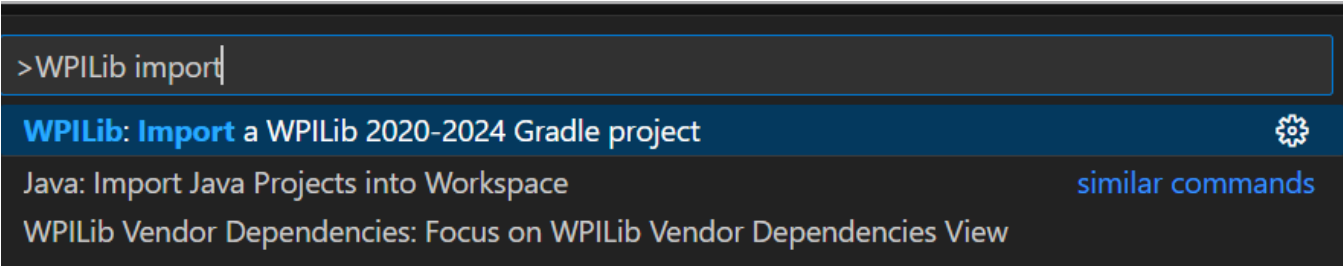


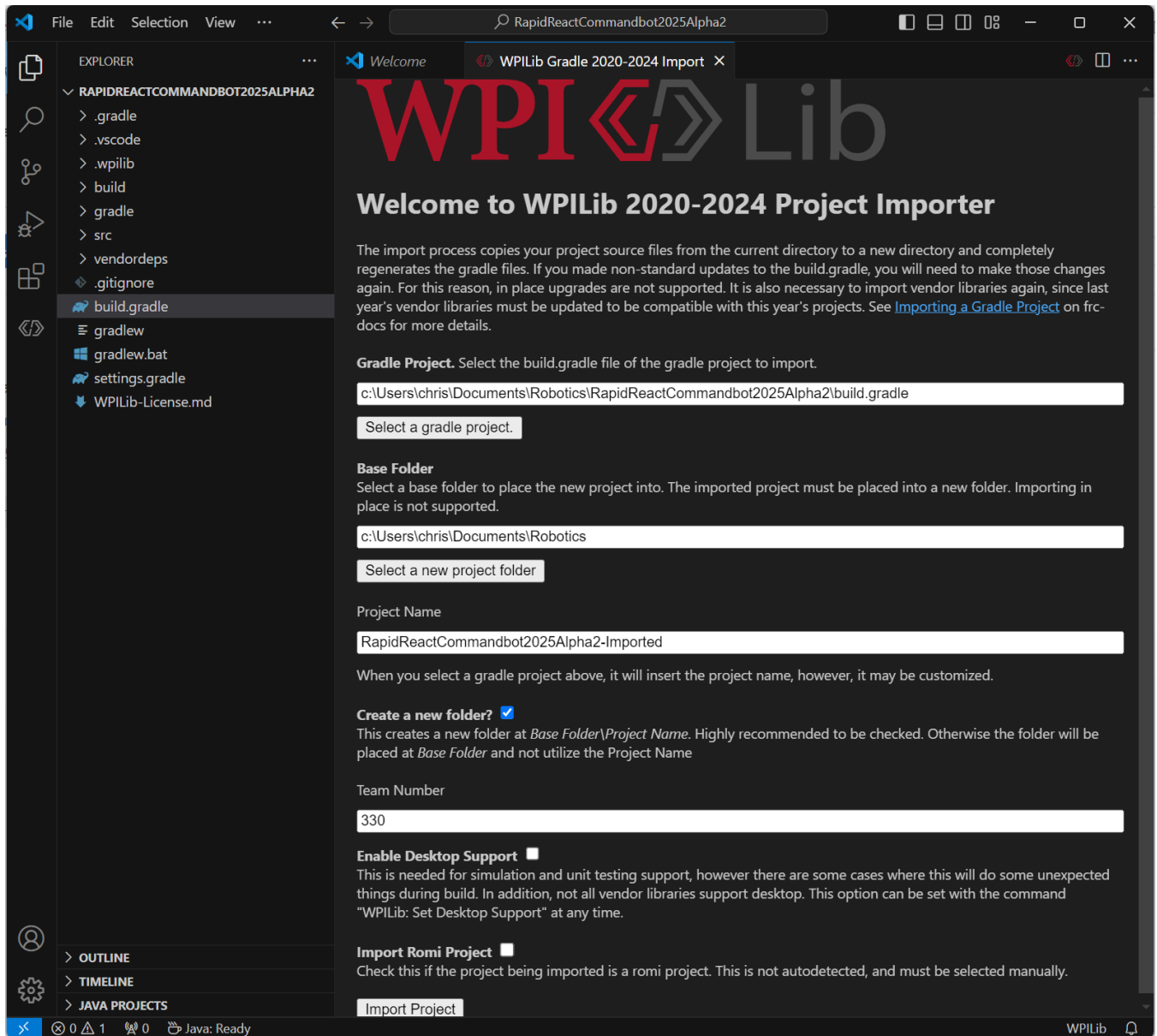
When you open a previous year's project, you will be prompted to import that project. Click **yes**.

打开上一年度的项目时，系统会提示导入该项目。单击 **yes**。

Alternately, you can chose to import it from the menu. Press Ctrl+Shift+P and type "WPILib" or click the WPILib icon to locate the WPILib commands. Begin typing "Import a WPILib 2020-2025 Gradle project" and select it from the dropdown as shown below.

也可以选择从菜单导入。按 Ctrl+Shift+P 并输入“WPILib”，或单击 WPILib 图标以查找 WPILib 命令。开始输入“Import a WPILib 2020-2025 Gradle project”，然后从下拉列表中选择该命令，如下所示。





You'll be presented with the WPILib Project Importer window. This is similar to the process of creating a new project and the window and the steps are shown below. This window contains the following elements:

随后会显示 WPILib Project Importer 窗口。此过程与创建新项目类似，窗口及其操作步骤如下所示。该窗口包含以下元素：

1. **Gradle Project:** Selects the project to be imported. Users should select the build.gradle file in the root directory of the gradle project.

Gradle Project: 选择要导入的项目。用户应选择 Gradle 项目根目录中的 build.gradle 文件。

2. **Project Location:** This determines the folder in which the robot project will be located.

Project Location: 确定机器人项目所在的文件夹。

3. **Project Name:** The name of the robot project. This also specifies the name that the project folder will be given if the Create New Folder box is checked. This must be a different directory from the original location.

Project Name: 机器人项目的名称。如果选中 Create New Folder 复选框，这也会指定项目文件夹的名称。该目录必须与原位置不同。

4. **Create a New Folder:** If this is checked, a new folder will be created to hold the project within the

previously-specified folder. If it is *not* checked, the project will be located directly in the previously-specified folder. An error will be thrown if the folder is not empty and this is not checked.

Create a New Folder: 如果选中此项，将在之前指定的文件夹内创建一个新文件夹来存放项目。如果未选中，项目将直接位于之前指定的文件夹中。如果该文件夹非空且未选中此项，将抛出错误。

5. **Team Number:** The team number for the project, which will be used for package names within the project and to locate the robot when deploying code.

Team Number: 项目的队伍编号，用于项目中的 package 名称，并在部署代码时用于查找机器人。

6. **Enable Desktop Support:** If this is checked, simulation and unit test support is enabled. However, there are some cases where this will do some unexpected things. In addition, all vendor libraries need desktop support which not all libraries do.

Enable Desktop Support: 如果选中此项，将启用仿真和单元测试支持。不过，在某些情况下，该选项会产生一些非预期行为。此外，所有 vendor library 都需要支持桌面平台，而并非所有库都支持。

7. **Import Romi Project:** If this is checked, the project is imported using the Romi gradle template. This should only be checked for Romi projects.

Import Romi Project: 如果选中此项，将使用 Romi Gradle 模板导入项目。仅 Romi 项目应选中此项。

WARNING: Creating projects on OneDrive is not supported as OneDrive's caching interferes with the build system. Some Windows installations put the Documents and Desktop folders on OneDrive by default.

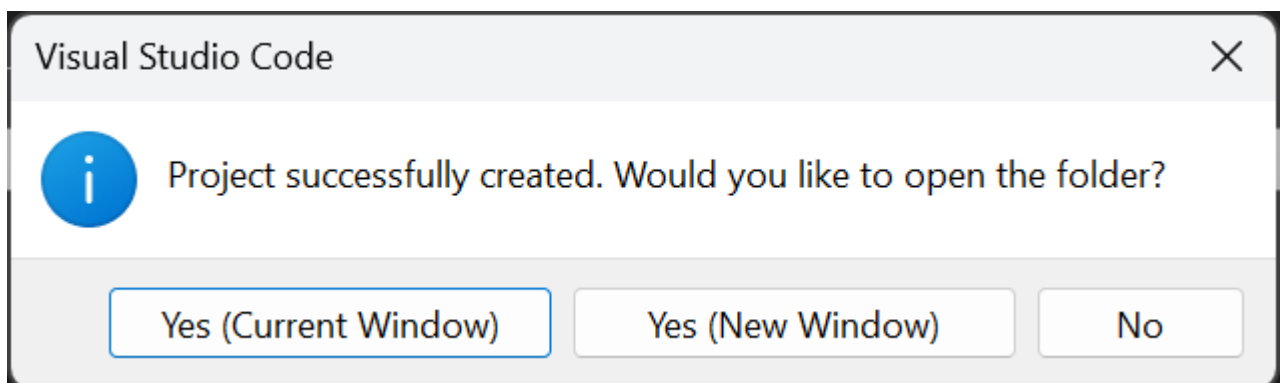
警告: 不支持在 OneDrive 上创建项目，因为 OneDrive 的缓存会干扰构建系统。某些 Windows 安装会默认将 Documents 和 Desktop 文件夹放在 OneDrive 上。

Click **Import Project** to begin the upgrade.

单击 **Import Project** 开始升级。

The gradle project will be upgraded and copied into the new project directory. You can then either open the new project immediately using the pop-up below or open it later using the Ctrl+O (or Command+O for macOS) shortcut.

Gradle 项目会完成升级并复制到新项目目录。随后，可以使用下方弹窗立即打开新项目，也可以稍后使用 Ctrl+O (macOS 上为 Command+O) 快捷键打开。





Do you trust the authors of the files in this folder?

Code provides features that may automatically execute files in this folder.

If you don't trust the authors of these files, we recommend to continue in restricted mode as the files may be malicious. See [our docs](#) to learn more.

`C:\Users\Joe\Documents\Robotics\RobotBuilderTestProjectCpp-Imported2022Alpha3`

☐

Trust the authors of all files in the parent folder 'Robotics'

Yes, I trust the authors
Trust folder and enable all features

No, I don't trust the authors
Browse folder in restricted mode

Click **Yes I trust the authors**.

单击 **Yes I trust the authors**。

3rd Party Libraries / 第三方库

It is necessary to update and re-import 3rd party libraries. See the 3rd Party Libraries section above for details.

必须更新并重新导入第三方库。有关详细信息，请参阅上文的“第三方库”章节。